Workgroup:	RPP
Internet-Draft:	draft-ietf-rpp-core-02 (editorial)
Published:	2 November 2026
Intended Status:	Standards Track
Expires:	6 May 2027
Authors:	M. Wullink	P. Kowalik
---	---	---
	SIDN Labs	DENIC

RESTful Provisioning Protocol (RPP)

Abstract

This document defines the mapping of the RESTful Provisioning Protocol (RPP) to the HTTP protocol. RPP is used for the provisioning and management of objects in a shared database. The RPP data model, consisting of Data Objects and their operations, is defined in [I-D.ietf-rpp-data-objects]. This document specifies how those Data Objects and operations are mapped to HTTP resources, methods, header fields, and status codes, and how RPP servers are discovered. The mapping is independent of the data format used to represent RPP messages.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 6 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 4
2. Terminology 5
3. Conventions Used in This Document 5
4. Notational Conventions 6
5. Mapping to EPP 6
6. Header Fields 6
6.1. Request Header Fields 6
6.2. Response Header Fields 7
7. Cross-Origin Resource Sharing 9
8. Redirection 9
9. Problem Detail Responses for Errors 10
10. Bootstrapping 11
11. Discoverability 12
11.1. Workflow 12
12. Versioning 13
12.1. Endpoints 13
12.2. Messages 13
12.3. Extensions 13
12.4. Profiles 14
13. Media Types 14
14. Profile Signalling 15
14.1. Rationale for Header Field Signalling 15
14.2. Header Signalling 16
14.3. Media Type Signalling 16
15. Process Signalling 18
16. Endpoints 19
16.1. HTTP Mapping Rules 19
16.1.1. Rule 1: Collection Path Segment 19

16.1.2. Rule 2: Uniform Interface Operations 20
16.1.3. Rule 3: Direct Access Sub-Resource Path Segment 20
16.1.4. Rule 4: Process Uniform Interface Operations 23
16.1.5. Rule 5: Extended Process Operations 24
16.1.6. Rule 6: Process Listing 24
16.1.7. Rule 7: Operation Parameters 25
16.2. Derived Endpoint Reference 25
16.3. Internationalized Domain Names (IDN) 28
16.4. Availability for Creation 29
16.5. Resource Information 30
16.6. Create Resource 31
16.7. Delete Resource 32
16.8. Update Resource 33
16.9. Processes 34
16.9.1. Relation to Object Representation 34
16.9.2. Response with Information about Created Process 35
16.9.3. Restore Resource 35
16.9.4. Renew Resource 36
16.9.5. Transfer Resource 37
16.9.5.1. Start 37
16.9.5.2. Status 38
16.9.5.3. Cancel 38
16.9.5.4. Reject 39
16.9.5.5. Approve 40
16.10. Messages 40
16.10.1. Retrieve 42
16.10.2. Acknowledge 43
17. Extension Framework 44
18. Result Codes 44
18.1. Mapping RPP Result Codes to HTTP Status Codes 45

19. Authentication and Authorisation 47
20. IANA Considerations 47
20.1. RPP Result Codes Registry 47
20.2. Link Relation Type 50
21. Internationalisation Considerations 50
22. Security Considerations 50
23. Change History 51
23.1. Version ietf-rpp-core-01 to ietf-rpp-core-02 51
23.2. Version ietf-rpp-core-00 to ietf-rpp-core-01 51
23.3. Version draft-wullink-rpp-core-05 to draft-ietf-rpp-core-00 52
23.4. Version 04 to 05 52
23.5. Version 03 to 04 53
23.6. Version 02 to 03 53
23.7. Version 01 to 02 53
23.8. Version 00 to 01 53
23.9. Version 00 (draft-rpp-core) to 00 (draft-wullink-rpp-core) 53
24. Acknowledgements 53
25. References 54
25.1. Normative References 54
25.2. Informative References 55
Authors' Addresses 56

1. Introduction

This document describes an Application Programming Interface (API) based on HTTP as defined in [HTTP] and the principles of [REST]. Conforming to the REST constraints is generally referred to as being "RESTful". Hence the API is dubbed: "RESTful Provisioning Protocol" or "RPP" for short.

The RPP API is designed to be used for the provisioning and management of objects in a shared database, such as domain names and their name servers (hosts).

The RPP data model, including the Data Objects, their data elements, and the operations defined on them, is specified independently of any protocol in [I-D.ietf-rpp-data-objects]. This document is the mapping of that data model to HTTP. It defines how the URL, HTTP method, header fields, and HTTP status code for each operation are derived from the Data Object definitions, as well as bootstrapping, discovery, versioning, media type negotiation, error reporting, and authentication.

This document does not define a data format. The representation of RPP messages in a specific data format, for example JSON, is defined in a separate representation specification that registers an RPP media type.

TODO Rename this document to better reflect its role as the HTTP mapping of the RPP data model.

2. Terminology

In this document the following terminology is used.

REST - Representational State Transfer ([REST]). An architectural style.

RESTful - A RESTful web service is a web service or API implemented using HTTP and the principles of [REST].

RESTful Provisioning Protocol or RPP - The protocol described in this document.

URL - A Uniform Resource Locator as defined in [RFC3986].

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

RPP client - An HTTP user agent performing an RPP request

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

JWT - JSON Web Token as defined in [RFC7519].

RPP media type - A media type, defined in a representation specification, that carries RPP messages in a specific data format.

Label Generation Ruleset (LGR) - A set of rules defining the valid labels for a registry, including the permitted repertoire, contextual rules, and, where applicable, variant mappings. Also historically known as IDN tables [IDN-Tables].

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

All
example requests assume an RPP server is available on the standard HTTPS port on host `rpp.example`. An authorisation token has been provided by an out-of-band process and is used by the client to authenticate each request.

4. Notational Conventions

This document uses the following terminology from Section 3 to specify syntax and parsing: Item, String, and Integer.

This document does not mandate any particular authority (host and port) or path prefix for an RPP server. The `rpp.example` authority and the `/rpp` path prefix used throughout the examples in this document are purely illustrative. All URLs used in examples are presumed to be relative to the value defined for the `base_url` field of the well-known discovery document.

5. Mapping to EPP

RPP is designed as an independent protocol and does not require an EPP server. RPP concepts such as transaction identifiers, result codes, and object attributes are defined in their own right and serve RPP purposes regardless of whether an EPP backend is present. However, compatibility with EPP is preserved to a great extent. Implementers with no prior EPP experience are able to implement RPP based solely on this specification.

Some RPP concepts are functionally similar to EPP concepts, but they are not directly derived from EPP and MAY have different semantics. To avoid confusion, RPP elements SHOULD NOT use an `EPP` prefix or suffix. For implementers who operate an EPP backend and need to bridge RPP requests to EPP commands, a separate RPP-to-EPP mapping document [TODO REF] is provided. Any extensions to RPP are not covered by that mapping document; the mapping of extension elements MUST be defined in the respective extension specification.

6. Header Fields

HTTP header fields defined by RPP MUST use the `RPP-` prefix and MUST be defined as Structured Fields [RFC9651], unless the syntax of the field value cannot be expressed as an Item, List or Dictionary. A header field definition that is not a Structured Field MUST state this explicitly. RPP also uses existing standard header fields, such as `Link` [RFC8288], which do not use the `RPP-` prefix.

6.1. Request Header Fields

This document defines the `RPP-Cltrid`, `RPP-Authorization` and `RPP-Profile` request header fields.

- `RPP-Cltrid`: A client-assigned transaction identifier. The client MUST include this header field in every request. It serves two independent purposes: as an idempotency key, allowing the server to detect and safely handle duplicate requests, and as an audit-trail identifier, enabling end-to-end correlation of a request across client and server logs.

The value MUST be unique per request. The `RPP-Cltrid` header field's value is an Item of type String.

- `RPP-Authorization`: The client MAY use this header field to send authorisation information in the format `<method> <authorization information>`, similar to the `Authorization` header field defined in Section 11.6.2. The `RPP-Authorization` header field is specific for each User Agent and MUST NOT be cached, as recommended by Section 16.4.2. The `<method>` indicates the type of authorisation being used. For EPP object authorisation information, for example the authorisation information used for domain names described in Section 2.6, a new `authinfo` method is defined. The `<authorization information>` defines the following comma-separated fields:
  - value (REQUIRED): Base64 encoded EPP password-based authorisation information. The content of the `value` field is case sensitive.
  - roid (OPTIONAL): A Roid as defined in [RFC5731], [RFC5733], and [RFC5730]. The roid is used to identify the object for which the authorisation information is provided. If the roid is not provided, then the server MUST assume that the authorisation
    information is linked to the object identified by the URL of the request.

Example request using the `RPP-Authorization` header field:

```
GET /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
RPP-Cltrid: ABC-12345
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu, roid=REG-123
```

- `RPP-Profile`: The client MUST use this header field to indicate the profile that is used in the request.

6.2. Response Header Fields

This document defines the `RPP-Svtrid`, `RPP-Cltrid`, `RPP-Code`, `RPP-Queue-Size` and `RPP-Profile` response header fields.

- `RPP-Svtrid`: A server-assigned transaction identifier. The server MUST include this header field in every response, providing a unique, server-side audit-trail reference for the processed request. Because this header field maps the EPP `svTRID` element, whose syntax differs from the String type defined in Section 3, `RPP-Svtrid` MUST NOT be defined as a Structured Field.

Example response including the `RPP-Svtrid` header field:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

- `RPP-Cltrid`: The server MUST echo the client transaction identifier from the request back to the client in this response header field. This allows the client to correlate responses to their originating requests. Because this header field maps the EPP `clTRID` element, whose syntax differs from the String type defined in Section 3, `RPP-Cltrid` MUST NOT be defined as a Structured Field.

Example response including the `RPP-Cltrid` header field:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
RPP-Cltrid: ABC-12345
RPP-Svtrid: XYZ-12345
RPP-Code: 01000
```

- `RPP-Code`: This header field contains the RPP result code indicating the outcome of the request. This header field MUST be included in all responses and MAY be used by the client for easy access to the result code, without having to parse the HTTP response content. The `RPP-Code` header field's value is an Item of type String.
- `RPP-Queue-Size`: Returns the number of unacknowledged messages in the client message queue. The server MAY include this header field in all RPP responses. The `RPP-Queue-Size` header field's value is an Item of type Integer and MUST be greater than or equal to 0.
- `Link`: When a uniform interface operation implicitly creates a process object as a side effect,
  the server MUST communicate the URL of the created process resource using the `Link` response header field [RFC8288] with the `rpp-process` relation type. If multiple process objects are created, the server MUST include one `Link` header field per created process resource, each with `rel="rpp-process"`.

Example response with the `Link` header field to indicate the URL of a process object created as a side effect of a domain create request:

```
HTTP/1.1 201 Created
Location: https://rpp.example/rpp/domainNames/foo.example
Link: <https://rpp.example/rpp/domainNames/foo.example/processes/\
   createProcesses/latest>; rel="rpp-process"; process="createProcess"; \
   processId="XYZ-12345"
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

- `RPP-Profile`: The server MUST use this header field to indicate the profile that is used in the response.

7. Cross-Origin Resource Sharing

RPP servers MAY be accessed directly by browser-based clients. To support such clients, RPP servers SHOULD implement Cross-Origin Resource Sharing (CORS) as defined in [FETCH].

When responding to a CORS preflight (OPTIONS) request, the server SHOULD include the following in its response:

- `Access-Control-Allow-Methods`: The HTTP methods supported for the requested endpoint.
- `Access-Control-Allow-Headers`: All request header fields defined in this specification in addition to `Content-Type` and `Accept`.
- `Access-Control-Expose-Headers`: All request and response header fields defined in this specification.

The server MUST allow every RPP media type it supports to be used in the `Content-Type` and `Accept` header fields of cross-origin requests.

8. Redirection

An RPP server, or an HTTP intermediary, can redirect a request to another URL. Depending on the status code, an HTTP client can change the method of the request when following a redirect, for example from POST to GET, which may break RPP processing. Therefore, as recommended by [RFC9205], this section defines how redirects are handled by RPP.

An RPP server MUST use only the 307 (Temporary Redirect) or 308 (Permanent Redirect) status code (Section 15.4.8 and Section 15.4.9) when redirecting a request, because these status codes do not allow the client to change the request method. A redirect response MUST include a `Location` header field containing an absolute URL using the `https` scheme.

An RPP client MAY follow a 307 or 308 redirect response. When following a redirect, the client MUST send the same request content and the same header fields as in the original request, including the `RPP-Cltrid` header field, so that the redirected request is the same request for the purpose of idempotency. The client MUST NOT send the `Authorization` and `RPP-Authorization` header fields to an origin that differs from the origin of the original request, unless the client knows that the target origin is an alias of the origin of the original

request, or the client has credentials that are valid for the new target origin. In the latter case the client uses the credentials for the new target origin, which can differ from the credentials used for the original request.

An RPP client MUST NOT follow a response with any other redirection status code, for example because an HTTP intermediary generated it, and MUST treat such a response as a protocol error.

9. Problem Detail Responses for Errors

When an error occurs that prevents processing of the requested action, an RPP server MUST respond using a Problem Detail document [RFC9457] detailing what went wrong, or what was not acceptable to the server. The `type` field MUST be the `urn:ietf:params:rpp:error` URN. The `status` field MUST reflect the HTTP status code. The document MUST contain an `errors` element, as a list of objects detailing individual errors.

This document consists of the following fields:

`type` (required, string) This field MUST always contain the fixed string value `urn:ietf:params:rpp:error` to indicate that this is an RPP Problem Detail response.
`title` (required, string) A short human-readable description of the Problem Detail type.
`errors` (optional, array of error objects) MUST contain objects detailing information about the specific errors that occurred, including optional references to which values in the original request were not acceptable to the server.

The `error` object consists of the following fields:

`type` (required, string) This field SHOULD be a URI that identifies the error type. This URI MAY be dereferenceable to obtain human-readable documentation about the error type.
`result` (required, string) This field MUST contain the RPP result code associated with the error.
`paths` (optional, list of strings) The JSONPath [RFC9535] to the value(s) in the request that caused the error. This field MAY be omitted if no specific value in the request can be identified as the cause of the error.
`reason` (required, string) A human-readable detailed description of the error.

RPP specifications and/or extensions MAY add extension fields to the error object, to convey additional information about the causes of the error. For example, to indicate the account balance on a billing failure, the following Problem Detail response could be used:

```
{
  "type": "urn:ietf:params:rpp:error",
  "title": "RPP Error",
  "errors": [{
    "type": "https://rpp.example/problems/billing/billing-failure",
    "result": "02104",
    "reason": "Not enough balance on account to create domain",
    "balance": 10.0,
    "action_cost": 25.0
  }]
}
```

Problem Detail response containing multiple errors for a domain create request using an invalid domain name (_$.example), JSONPaths are specified in the error objects to indicate which value in the request caused the error:

```
{
  "type": "urn:ietf:params:rpp:error",
  "title": "RPP Error",
  "errors": [{
    "type": "https://rpp.example/problems/domains/domain-syntax",
    "result": "02005",
    "paths": ["$.domain.name"],
    "reason": "Invalid character(s) in domain name"
  },
  {
    "type": "https://rpp.example/problems/domains/domain-allowed-length",
    "result": "02004",
    "paths": ["$.domain.name"],
    "reason": "Domain name length must be between 3 and 63 characters"
  }]
}
```

10. Bootstrapping

The server MUST provide a mechanism for clients to discover the location of the RPP server, two methods are defined for this purpose, using the IANA registry for RPP servers or using DNS-based bootstrapping. The server MUST provide at least one of these methods, and MAY provide both methods. The client MUST use either the IANA registry for RPP servers or a DNS lookup using an HTTPS resource record as defined in [RFC9460]. The client MUST be able to handle the case where the chosen method does not return a valid RPP server location, and MUST be able to use the other method as a fallback to discover the RPP server location.

The format and procedure for adding an RPP server to the IANA registry is defined in the IANA Considerations section below. If the server uses both the IANA registry and DNS-based bootstrapping, then both methods MUST return the same location for the RPP server.

For DNS-based bootstrapping, an RPP server MUST publish an HTTPS resource record in a `_rpp` child zone for each DNS zone that is managed by the RPP server. The owner name of the HTTPS resource record MUST be the managed zone name itself, prepended with the `_rpp.` label (i.e., `_rpp.<zone>`).

If multiple HTTPS resource records are returned, the client MUST process them according to the priority rules defined in [RFC9460]. If multiple records have the same SvcPriority, the client SHOULD select one based on local policy.

The client MUST construct the URL for the well-known endpoint of the RPP server as follows, using the `TargetName` and `port` SvcParams from the HTTPS resource record:

- `https://<TargetName>:<port>/.well-known/rpp` when the `port` SvcParam is present and not 443
- `https://<TargetName>/.well-known/rpp` when the `port` SvcParam is absent or is 443

Example HTTPS resource record for an RPP server for the TLD `example` running HTTPS on port 443 at `rpp.example.`:

```
$ORIGIN example.
_rpp IN HTTPS 1 rpp.example. alpn=h2,h3
```

In this example, the well-known endpoint URL is `https://rpp.example/.well-known/rpp`.

Example HTTPS resource record for an RPP server using a different TargetName (rpp-svr2.registry.example) and a non-standard port (8443) for the TLD `example`:

```
$ORIGIN example.
_rpp IN HTTPS 1 rpp-svr2.registry.example. alpn=h2,h3 port=8443
```

In this example, the well-known endpoint URL is `https://rpp-svr2.registry.example:8443/.well-known/rpp`.

11. Discoverability

An RPP client MUST be able to discover the capabilities of the RPP server before performing any provisioning operations. The Discovery Data Object defined in [I-D.ietf-rpp-data-objects] MUST be mapped to the well-known endpoint at `/.well-known/rpp`, and MUST return a representation describing the capabilities of the RPP server. The well-known endpoint MUST be accessible, using the HTTP GET method, without authentication and MUST return the 200 (OK) status code if the request was successful.

11.1. Workflow

The steps for a typical workflow of provisioning an object using RPP without knowing the location and capabilities of the server are as follows, the first three steps are optional, the client can choose to skip any of these steps if it already has the required information from a previous interaction or configuration.

1. Bootstrap (optional): The client discovers the location of the RPP server by looking up the IANA registry for RPP servers or by performing a DNS HTTPS lookup as defined in [RFC9460].
2. Discover capabilities (optional): The client retrieves the capabilities of the RPP server by sending a GET request to the well-known endpoint at `/.well-known/rpp`.

3. Extract RPP URLs (optional): The client extracts the base URL and endpoint URL templates from the discovery response, and uses this information to construct the URLs for the desired operations.
4. Perform provisioning operations: The client performs provisioning operations by sending HTTP requests to the appropriate endpoint URLs, using the HTTP method and request content as required by the specific operation.

12. Versioning

RPP operator MAY support more than one version of RPP. The versioning scheme uses the Semantic Versioning format defined in [SemVer], where major version number is used to indicate breaking changes. In this case of incompatible versions the operator MUST deploy them on distinct `base_url` and announce them in discovery document.

The client MUST indicate the version of RPP it is using by including an RPP media type with a `version` parameter in the `Accept` header field of each request, for example: `application/rpp+json; version=1.0`. The server MUST include the media type used in the request in its response to the client. The server MUST reject requests using an unsupported version of RPP, and MUST return a 406 (Not Acceptable) status code.

The following additional RPP elements include versioning support:

- Messages: A request and response message MUST include the version of the RPP API it is compatible with.
- Extensions: RPP extensions MUST include the version of the RPP API they are compatible with.
- Profiles: RPP profiles MUST include the version of the RPP API they are compatible with.
- Media types: RPP media types MUST include the version of the RPP API they are compatible with.
- Result codes: RPP result codes may be added by extensions or updates to the core RPP specification.

12.1. Endpoints

The `base_url` element of the RPP Discovery response may include a version segment indicating the version of the RPP API supported by the server. A server MAY publish a `base_url` such as `https://rpp.example/rpp/v1/` (the `rpp.example` authority and `/rpp/v1` prefix being illustrative only).

12.2. Messages

The `version` element of the RPP request and response messages MUST include the version of the RPP API that the message is compatible with. The server MUST reject requests with a version that is not supported by the server, and MUST return an RPP Client error code.

12.3. Extensions

The RPP server MUST include all supported extensions and the version for each extension in the RPP Discovery response. The client MUST use this version information to determine which extensions are supported by the server, and to ensure that it uses the correct version of the extension when making requests to the server. The server MUST reject requests

requiring extensions or unsupported versions of an extension that are not supported by the server. The set of extensions required by the client MUST be signalled by the client by using the correct `profile`, see Section 12.4.

12.4. Profiles

The RPP server MUST provide the set of supported profiles in the RPP Discovery response in the `profiles` element. The client can then request the appropriate profile from the server, using the `profiles` endpoint. The client MUST use this version information to determine which profiles are supported by the server, and to ensure that it uses the correct version of the profile when making requests to the server. A request using a profile MUST include the version of the profile. The server MUST reject requests using profiles with a version that is not supported by the server, and MUST return an RPP Client error code.

13. Media Types

RPP data can be transmitted using different representation formats, for example JSON or YAML. This document does not define a representation. Each representation is defined in a separate
specification that registers an RPP media type for that representation, for example `application/rpp+json`. Each such specification also documents the security considerations of its underlying data format. Every RPP media type MUST support the `profile` and `profile-version` parameters defined in Section 14.3, so that profile signalling is identical for all representations.

An RPP client and an RPP server indicate and negotiate the media type of a message using the `Content-Type` (Section 8.3) and `Accept` (Section 12.5.1) header fields, as follows.

Requests:

- An RPP client MUST include an `Accept` header field in every request, including requests without content. Each media range in the `Accept` header field MUST identify an RPP media type the client is able to process in the response. The client MAY list more than one media range and MAY use quality values to indicate preference.
- An RPP client sending a request MUST include a `Content-Type` header field identifying the requested RPP media type, even when there is no content.
- The media type indicated in the `Content-Type` header field MUST also be acceptable according to the `Accept` header field of the same request.

Server processing:

- The RPP server MUST determine the data format of request content only from the `Content-Type` header field and MUST NOT infer it from the content. If the content does not conform to the data format of the indicated media type, the server MUST reject the request with the 400 (Bad Request) status code.
- If the server does not support the media type indicated in the `Content-Type` header field, the server MUST reject the request with the 415 (Unsupported Media Type) status code.
- The RPP server MUST select the media type for the response from the `Accept` header field using proactive negotiation as described in Section 12.5.1. If the server supports

none of the listed media types, the server MUST reject the request with the 406 (Not Acceptable) status code.

- If the media type indicated in the `Content-Type` header field is not acceptable according to the `Accept` header field, the server MUST reject the request with the 400 (Bad Request) status code.
- In all rejection cases above, the server MUST include the `RPP-Code` header field and a Problem Detail response as described in Section 9.

Responses:

- An RPP server MUST include a `Content-Type` header field in every successful response. The media type MUST be one listed in the `Accept` header field of the request.
- Problem Detail responses use the `application/problem+json` media type [RFC9457].
- An RPP server MUST include `Accept` in the `Vary` header field Section 12.5.5 of every response whose content depends on the negotiated media type or profile.
- An RPP server SHOULD include the `X-Content-Type-Options: nosniff` header field [FETCH] in every response. Omitting it allows a browser-based RPP client to interpret a response as a media type other than the one indicated in `Content-Type`, which can expose registration data to scripts from another origin.

Client processing:

- The RPP client MUST determine the data format of response content only from the `Content-Type` header field and MUST NOT infer it from the content.
- Except for Problem Detail responses, if the media type of a response was not listed in the `Accept` header field of the request, the client MUST NOT process the content as an RPP response and MUST treat the response as a protocol error.

14. Profile Signalling

This document describes two distinct methods for signalling the profile used in an RPP request or response, the first method uses a dedicated HTTP header field, the second method uses media type parameters. The two methods MUST NOT be used simultaneously in a single request or response. If both methods are used in a single request or response, then the server MUST return an HTTP error response and include a Problem Detail response in the content.

14.1. Rationale for Header Field Signalling

This section is not normative. Signalling the profile using a dedicated HTTP header field is preferable to signalling it using media type parameters, for the following reasons:

- Protocol level versus content level: the profile, and the RPP version it implies, determines how the whole HTTP message is interpreted, including URL paths, query parameters, and HTTP header fields, which can all evolve over time and need agreement between client and server. A media type only describes the format of the content. Signalling at the protocol level, in a header field, fits this scope better than signalling in a property of the content.
- Independence of the representation format: media type parameters have to be mandated for, and implemented identically in, every RPP media type registered for

every representation format. A header field is independent of the representation format, so a new representation format does not need to define any profile signalling and the profile signalling does not need to be changed when the media type handling changes.

- Messages without content: `Content-Type` is only meaningful for a message that has content. Requests such as GET and DELETE, and responses without content, would otherwise have to carry a `Content-Type` header field only to signal the profile. A header field defined by RPP can be present in every request and response.
- Separation of concerns: with a dedicated header field, media types are used only to negotiate the data format of the content, and the profile is negotiated independently of it.

> TODO: the working group has not yet decided on the signalling method to use, the two methods are both described until that decision is made.

14.2. Header Signalling

The client MUST use the `RPP-Profile` header field to indicate the name of the profile that is to be used for the request. The value of this header field is a Dictionary as defined in Section 3.2. The Dictionary MUST contain a `profile` member whose value uniquely identifies a profile, for example `urn:ietf:params:rpp:profile:example-profile`, and this member MUST carry a `version` parameter indicating the version of the profile. If the server does not support the indicated profile or version, then the server MUST return an HTTP error response and include a Problem Detail response in the content.

The ABNF for the `RPP-Profile` header field value is as follows:

```
profile-header = "profile" "=" profile-name ";" OWS "version" "=" version
profile-name   = token
version        = 1*DIGIT "." 1*DIGIT
```

Example:

```
GET /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
RPP-Profile: profile=urn:ietf:params:rpp:profile:example-profile;version=1.0
RPP-Cltrid: ABC-12345
```

14.3. Media Type Signalling

An RPP client and an RPP server select the profile that applies to a message using the media type parameters defined in this section, on top of the `Content-Type` and `Accept` header field handling defined in Section 13. The profiles supported by an RPP server are announced in the RPP Discovery document (see Section 11).

The media type parameters are defined as follows:

- `profile`: The value of this parameter MUST uniquely identify the profile, for example `urn:ietf:params:rpp:profile:example-profile`.
- `profile-version`: The value of this parameter MUST indicate the version of the profile.

The ABNF for the media type parameters is as follows:

```
profile-parameter = "profile" "=" profile-name ";" OWS "profile-version" "="
version
profile-name      = token
version           = 1*DIGIT "." 1*DIGIT
```

Requests:

- Each media range in the `Accept` header field of a request MUST include the `profile` and `profile-version` parameters of a profile the client is able to process in the response. The client MAY list more than one profile and MAY use quality values to indicate preference.
- The media type in the `Content-Type` header field of a request with content MUST include the `profile` and `profile-version` parameters of the profile that the content conforms to.
- The profile indicated in the `Content-Type` header field MUST also be listed in the `Accept` header field of the same request.

Server processing:

- The RPP server MUST determine the profile of request content only from the `Content-Type` header field and MUST NOT infer it from the content, and MUST process the content according to that profile.
- If the server does not support the indicated profile or profile version, the server MUST reject the request with HTTP status code 415 (Unsupported Media Type).
- The RPP server MUST select the profile for the response from the `Accept` header field using proactive negotiation as described in Section 12.5.1. If the server supports none of the listed profiles, the server MUST reject the request with HTTP status code 406 (Not Acceptable).
- If the profile in the `Content-Type` header field is not listed in the `Accept` header field, the server MUST reject the request with HTTP status code 400 (Bad Request).
- If a request does not include profile parameters, the server MUST apply the default profile advertised in the RPP Discovery document (see Section 11).
- The requested profile may include extensions that include additional HTTP header fields or query parameters that the server needs to understand and process correctly.
- In all rejection cases above, the server MUST include the `RPP-Code` header field and a Problem Detail response as described in Section 9.

Responses:

- The media type in the `Content-Type` header field of a successful response that contains content MUST include the `profile` and `profile-version` parameters of the profile the

server applied, and that profile MUST be one listed in the `Accept` header field of the request.
- Problem Detail responses do not carry profile parameters.

Client processing:

- The RPP client MUST determine the profile of response content only from the `Content-Type` header field and MUST interpret the content according to that profile.
- If that profile or profile version was not listed in the `Accept` header field of the request, the client MUST NOT process the content as an RPP response and MUST treat the response as a protocol error.
- The requested profile may include extensions that include additional HTTP header fields or query parameters that the client needs to understand and process correctly.

Example of a request and the corresponding response header fields, using `application/rpp+json` for illustration:

```
GET /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json; profile="urn:ietf:params:rpp:profile:example-
profile"; profile-version="1.0"
RPP-Cltrid: ABC-12345

HTTP/1.1 200 OK
Content-Type: application/rpp+json;
profile="urn:ietf:params:rpp:profile:example-profile"; profile-version="1.0"
Vary: Accept
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

15. Process Signalling

When a server creates a process resource as a side effect of a uniform interface operation, it signals this to the client using the `Link` response header field [RFC8288] with `rel="rpp-process"`. The following target attributes are defined for this relation type:

- `process`: (REQUIRED) The process object identifier of the created process (e.g. `process="transferProcess"`).
- `processId`: (OPTIONAL) The server-assigned persistent identifier of the created process instance (e.g. `processId="XYZ-12345"`). If the server assigns a persistent identifier, this attribute MUST be included.
- `latest`: (OPTIONAL) The boolean value `true`, indicating that the target URL is the latest process created of a given type. The target URL MAY use the `latest` mnemonic rather than a specific process identifier. There MUST NOT be more than one process of a given type with this attribute set to `true`.

If multiple process objects are created, the server MUST include one `Link` header field per created process resource.

Example:

```
HTTP/1.1 201 Created
Link: <https://rpp.example/rpp/domainNames/foo.example/processes/\
   transferProcesses/XYZ-12345>; rel="rpp-process"; \
   process="transferProcess"; processId="XYZ-12345"; latest="true"
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16. Endpoints

Endpoints are described using URI Templates [RFC6570] relative to a discoverable base URL. The base URL, including its authority and any path prefix, is not defined by this specification; it is deployment-defined and discoverable by the client using the `base_url` field of the well-known discovery document, as described in Section 11. The URL paths used in the rules and examples below (e.g. `/{collection}/{id}`) are relative to the discovered base URL; the `rpp.example` authority and `/rpp` path prefix used in examples throughout this document are illustrative only and carry no normative meaning.

16.1. HTTP Mapping Rules

All RPP endpoints are derived mechanically from the Data Object definitions in [I-D.ietf-rpp-data-objects]. No endpoint URL or HTTP method related to processing of provisioning objects is defined independently of a corresponding Data Object. The rules in this section MUST be applied to determine the URL path and HTTP method for any operation.

16.1.1. Rule 1: Collection Path Segment

Each Data Object has a stable, `Identifier` (e.g. `domainName`, `contact`, `host`). The URL path segment for a collection of such objects MUST be derived by applying the `plural()` function to the object's `Identifier`. The `plural()` function appends an "s" or "es" to the identifier as per English language rules. Any irregular plural version MUST be defined in the data object specification.

```
{collection} = plural(dataObject.identifier)
```

Examples derived from current data object identifiers:

Data Object `Identifier`	`plural()` result	URL collection segment
domainName	domainNames	/domainNames
contact	contacts	/contacts
host	hosts	/hosts
organisation	organisations	/organisations
message	messages	/messages
profile	profiles	/profiles

Table 1

16.1.2. Rule 2: Uniform Interface Operations

The uniform interface operations defined in the RPP data object specification map to HTTP methods and URL paths as follows. `{collection}` is derived per Rule 1. `{id}` is the unique identifier value of the specific object instance.

Operation `Identifier`	HTTP Method	URL path
create	POST	/{collection}
read	GET	/{collection}/{id}
query	GET	/{collection}
update	PUT or PATCH	/{collection}/{id}
delete	DELETE	/{collection}/{id}
availability	HEAD or GET	/{collection}/{id}/availability

Table 2

16.1.3. Rule 3: Direct Access Sub-Resource Path Segment

A data element whose `Direct Access` flag is set to `true` in its Data Object definition is additionally exposed as a sub-resource, nested under the URL of the resource instance that contains it. This rule applies uniformly and recursively: the containing resource instance MAY itself be a Direct Access sub-resource of a further-enclosing resource.

The fixed path segment for such a sub-resource MUST be derived from the Direct Access data element's own `Identifier` - not from the `Identifier` of the associated object type the element references.

If the Direct Access element's cardinality is 0-1 or 1, the element's own `Identifier` MUST be used as the path segment, unpluralised; the resulting path already addresses a single instance unambiguously.

```
{direct-access-path} = {container-path} "/" directAccessElement.identifier
```

If the Direct Access element's cardinality is greater than 1, the `plural()` function MUST be applied to the element's `Identifier` to derive the path segment, and an individual associated object instance MUST be additionally addressed by appending its Unique Identifier value as a further path segment, `{unique-id}`. [I-D.ietf-rpp-data-objects] requires that any object type referenced by a Direct Access element of cardinality greater than 1 define a Unique Identifier for exactly this purpose.

```
{direct-access-path} = {container-path} "/"
plural(directAccessElement.identifier) "/" {unique-id}
```

Applying Rule 3 recursively from the top-level Data Object down to every Direct Access element defined in [I-D.ietf-rpp-data-objects] yields the following paths (`{id}` denotes the Unique Identifier value of the resource instance immediately to its left; it is instantiated per resource type as shown in the Derived Endpoint Reference below):

Container path	Direct Access element Identifier	Cardinality	Resulting path
/domainNames/{id}	processes	0-1	/domainNames/{id}/processes
/contacts/{id}	processes	0-1	/contacts/{id}/processes
/hosts/{id}	processes	0-1	/hosts/{id}/processes
/organisations/{id}	processes	0-1	/organisations/{id}/processes
/organisations/{id}	users	0+	/organisations/{id}/users/{id}
/organisations/{id}/users/{id}	processes	0-1	/organisations/{id}/users/{id}/processes
/{collection}/{id}/processes	transferProcess	0+	/{collection}/{id}/processes/transferProcesses/{process-id}
/{collection}/{id}/processes	renewProcess	0+	/{collection}/{id}/processes/renewProcesses/{process-id}
/{collection}/{id}/processes	restoreProcess	0+	/{collection}/{id}/processes/restoreProcesses/{process-id}
/{collection}/{id}/processes	createProcess	0+	/{collection}/{id}/processes/createProcesses/{process-id}

Table 3

`{collection}` in the last four rows is derived per Rule 1 and stands for any owner Data Object's collection segment (e.g. `domainNames`, `contacts`); the `Processes` element and its nested Process Object elements are defined once, generically, and Direct Access sub-resources derive identically regardless of the owner Data Object type. `{process-id}` is the Process Object's Unique Identifier value; Rule 4 defines the `latest` mnemonic as an additional way to address it.

The remaining rules in this section (Rules 4 through 6) apply Rule 3 specifically to Process Objects and describe how they are further exposed and interacted with.

16.1.4. Rule 4: Process Uniform Interface Operations

The four uniform interface operations from Rule 2 apply to Process Objects at the path derived per Rule 3. A server MAY assign a unique identifier `{process-id}` to each process instance and make it addressable by this identifier. The fixed keyword `latest` is used to address the most recent process instance when no specific process `{process-id}` is known or assigned.

Operation Identifier	HTTP Method	URL path
create	POST	/{collection}/{id}/processes/{process-collection}
read	GET	/{collection}/{id}/processes/{process-collection}/latest
read (specific instance)	GET	/{collection}/{id}/processes/{process-collection}/{process-id}
delete	DELETE	/{collection}/{id}/processes/{process-collection}/latest
delete (specific instance)	DELETE	/{collection}/{id}/processes/{process-collection}/{process-id}

Table 4

A started process MAY create a resource accessible using both the `latest` mnemonic and a server-assigned `{process-id}`. If the server exposes any access to a process instance, access to the most recent instance via `latest` MUST be supported.

When a process is created and immediately completed by the server, a 201 Created response MAY still be provided with a `Location` header field pointing to the created process resource.

If the server chooses not to expose any persistent process resource, it MUST return 200 OK instead of 201 Created.

The operations using the DELETE method MUST NOT accept request content, because the semantics of content in a DELETE request are not defined Section 9.3.5 and may cause deployment issues. If request content is present for any operation using the DELETE method, the server MUST reject the request with an appropriate error response.

This means that the `delete` operation (HTTP DELETE method) is not extensible, if the server needs to support additional data for a special `delete` operation, then a new process MUST be defined for this purpose.

16.1.5. Rule 5: Extended Process Operations

Operations on a Process Object beyond the uniform interface (e.g. `transferApprove`, `transferReject`, `report`) are mapped to sub-resources of a specific process instance. The operation's `Identifier` is used unchanged as the final path segment. The HTTP method for all such extended operations MUST be `POST`.

Operation `Identifier`	**HTTP Method**	**URL path**
`{operationIdentifier}`	`POST`	`/{collection}/{id}/processes/{process-collection}/latest/{operationIdentifier}`
`{operationIdentifier}` (specific instance)	`POST`	`/{collection}/{id}/processes/{process-collection}/{process-id}/{operationIdentifier}`

Table 5

The operation `Identifier` MUST be used as-is with no transformation. The following examples are derived directly from the operation identifiers defined in the RPP data object specification:

Operation `Identifier`	**Derived URL path (relative to `domainNames/foo.example`)**
`transferApprove`	`/domainNames/foo.example/processes/transferProcesses/latest/transferApprove`
`transferReject`	`/domainNames/foo.example/processes/transferProcesses/latest/transferReject`

Table 6

16.1.6. Rule 6: Process Listing

A server MAY implement a listing facility for processes. If implemented, the following URL structures MUST be used by the client.

To retrieve all process instances of a given process type:

```
GET /{collection}/{id}/processes/{process-collection}
```

To retrieve all process instances across all process types for an object:

```
GET /{collection}/{id}/processes
```

A server MAY choose not to implement these endpoints, in which case it MUST return 404 Not Found or 501 Not Implemented.

16.1.7. Rule 7: Operation Parameters

An operation parameter defined in [I-D.ietf-rpp-data-objects], such as `includeElements` MUST be included in the query component of the request URL, for every HTTP method, and MUST NOT be carried in the request content. This rule also applies to operation parameters added to an operation by an extension.

The query component is derived as follows:

- The parameter name MUST be the `Identifier` of the operation parameter, unchanged.
- The parameter value MUST be the value of the data element, serialised as defined for its Data Type, only primitive types SHOULD be used.
- An operation parameter with a cardinality of `0+` or `1+` MUST be repeated in the query component, once for each value.
- Multiple operation parameters are allowed, no specific ordering is required.

Example request for a domain name, using the `includeElements` operation parameter to request an additional data element:

```
GET /domainNames/foo.example?includeElements=authInfo HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
RPP-Cltrid: ABC-12345
```

The server MUST reject the request with the 400 (Bad Request) status code, the `RPP-Code` header field, and a Problem Detail response as described in Section 9 if:

- The query component contains a parameter name that is not an operation parameter of the requested operation or of a supported extension;
- A parameter with a cardinality of `0-1` or `1` occurs more than once;
- A parameter value does not conform to the Data Type of the operation parameter.

16.2. Derived Endpoint Reference

The following table lists all current RPP endpoints, each derived by applying the rules above to the relevant data object and operation identifiers. The following table is non-normative.

Operation	HTTP Method	URL path
Domain Name: read	GET	/domainNames/{name}
Domain Name: create	POST	/domainNames
Domain Name: update	PUT or PATCH	/domainNames/{name}
Domain Name: delete	DELETE	/domainNames/{name}
Domain Name: availability	HEAD or GET	/domainNames/{name}/availability
Contact: read	GET	/contacts/{id}
Contact: create	POST	/contacts
Contact: update	PUT or PATCH	/contacts/{id}
Contact: delete	DELETE	/contacts/{id}
Contact: availability	HEAD or GET	/contacts/{id}/availability
Host: read	GET	/hosts/{hostName}
Host: create	POST	/hosts
Host: update	PUT or PATCH	/hosts/{hostName}
Host: delete	DELETE	/hosts/{hostName}
Host: availability	HEAD or GET	/hosts/{hostName}/availability
Organisation: read	GET	/organisations/{id}
Organisation: create	POST	/organisations
Organisation: update	PUT or PATCH	/organisations/{id}
Organisation: delete	DELETE	/organisations/{id}
User: read	GET	/organisations/{id}/users/{id}
User: create	POST	/organisations/{id}/users

Operation	HTTP Method	URL path
User: update	PUT or PATCH	/organisations/{id}/users/{id}
User: delete	DELETE	/organisations/{id}/users/{id}
Discovery: read	GET	/.well-known/rpp
Message: read	GET	/messages/{id}
Message: create	POST	/messages
Message: query	GET	/messages
Message: delete	DELETE	/messages/{id}
Transfer: read	GET	/{collection}/{id}/processes/transferProcesses/latest
Transfer: create	POST	/{collection}/{id}/processes/transferProcesses
Transfer: delete	DELETE	/{collection}/{id}/processes/transferProcesses/latest
Transfer: transferApprove	POST	/{collection}/{id}/processes/transferProcesses/latest/transferApprove
Transfer: transferReject	POST	/{collection}/{id}/processes/transferProcesses/latest/transferReject
Transfer: list	GET	/{collection}/{id}/processes/transferProcesses
Restore: read	GET	/{collection}/{id}/processes/restoreProcesses/latest
Restore: create	POST	/{collection}/{id}/processes/restoreProcesses
Restore: update	PUT or PATCH	/{collection}/{id}/processes/restoreProcesses/latest
Restore: list	GET	/{collection}/{id}/processes/restoreProcesses
Renew: create	POST	/{collection}/{id}/processes/renewProcesses

Operation	HTTP Method	URL path
Renew: list	GET	/{collection}/{id}/processes/renewProcesses
Create: read	GET	/{collection}/{id}/processes/createProcesses/latest
Create: create	POST	/{collection}/{id}/processes/createProcesses
Create: list	GET	/{collection}/{id}/processes/createProcesses
Processes: list	GET	/{collection}/{id}/processes
Profile: read	GET	/profiles/{name}

Table 7

> TODO: The User Data Object derives the path `/organisations/{id}/users/{id}`, in which the placeholder `{id}` occurs twice. See https://github.com/ietf-wg-rpp/draft-ietf-rpp-data-objects/issues/196

All URL paths in the table above are relative to the discovered base URL, except the Discovery endpoint, which is located at `/.well-known/rpp` relative to the origin of the RPP server.

16.3. Internationalized Domain Names (IDN)

When an Internationalized Domain Name (IDN) is used to identify a Domain Name Data Object or Host Data Object instance in a URL, in particular as the `{id}` path segment described in Rule 2, the ASCII Compatible Encoding (ACE) A-label form of the name, as defined in [RFC5890], MUST be used. This requirement ensures that the resulting URL remains a valid URI as defined in [RFC3986], since the A-label form is composed exclusively of ASCII characters and therefore requires no percent-encoding or additional Unicode normalisation when used as a URL path segment.

The Unicode (U-label) form of an internationalized name MUST NOT be used to address a resource in a URL. A client MAY submit or receive the U-label form as a separate data element as part of a resource representation, but this Unicode representation of the name as a whole MUST NOT be used when constructing or matching a URL.

This is consistent with the use of IDN in the DNS, the actual DNS name is naturally represented using its A-label form. This also avoids ambiguity: the URL identifies the DNS domain name rather than a particular Unicode representation of it.

For example, to address the domain name whose U-label is `bücher.example`, a client MUST use the corresponding A-label, `xn--bcher-kva.example`, when constructing the `{id}` path segment:

```
GET /domainNames/xn--bcher-kva.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
RPP-Cltrid: ABC-12345
```

16.4. Availability for Creation

The Availability for Creation endpoint maps the Availability operation defined in [I-D.ietf-rpp-data-objects] and is used to check whether an object can be successfully provisioned. Two distinct methods are defined for checking the availability of provisioning of an object, the first method uses the HEAD method for a quick check to find out if the object can be provisioned. The second method uses the GET method to retrieve the Availability Object, which may contain additional information, for example about pricing or additional requirements to be able to provision the requested object.

When the client uses the HTTP HEAD method, the server MUST respond with an HTTP status code 200 (OK) if the object can be provisioned or with an HTTP status code 404 (Not Found) if the object cannot be provisioned.

When the client uses the HTTP GET method, the server MUST respond with an HTTP status code 200 (OK) if the object can be provisioned. The server MUST include the Availability Object in the response content.

If the object cannot be provisioned then the server MUST return an HTTP status code 404 (Not Found) and include the Availability Object in the response content.

Parameters added to the Availability operation by extensions, for example to request a certain currency as defined in the Registry Fee Extension [RFC8748], are mapped to query parameters as described in Section 16.1.7.

The server MUST respond with the same HTTP status code if the same URL is requested with HEAD and with GET.

```
- Request: HEAD|GET /{collection}/{id}/availability
- Request message: None
- Response message: Availability Object (GET only)
```

Example request for a domain name that is not available for provisioning:

```
HEAD /domainNames/foo.example/availability HTTP/1.1
Host: rpp.example
Accept: application/rpp+json
Authorization: Bearer <token>
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/1.1 404 Not Found
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
RPP-Cltrid: ABC-12345
RPP-Svtrid: XYZ-12345
RPP-Code: 01000
Content-Length: 0
```

Example request for an internationalized domain name, using the `uName` and `lgr` operation parameters to have the availability determined for the Unicode Name `bücher.example` against a specific LGR:

```
GET /domainNames/xn--bcher-kva.example/availability?\
   uName=b%C3%BCcher.example&lgr=example-latn-1.0 HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Type: application/rpp+json
Content-Language: en
RPP-Cltrid: ABC-12345
RPP-Svtrid: XYZ-12345
RPP-Code: 01000
Content-Length: 42
```

16.5. Resource Information

The Object Info request MUST use the HTTP GET method on a resource identifying an object instance (Rule 2, `read` operation). If the object has authorisation information attached then the client MUST send no content and include the `RPP-Authorization` header field. If the authorisation is linked to a database object the client MUST also include the roid in the `RPP-Authorization` header field. The client MAY also send content that includes the authorisation information, the client MUST then not use the `RPP-Authorization` header field.

Example request for an object not using authorisation information, using the `hostsFilter` operation parameter to only return delegated hosts:

```
GET /domainNames/foo.example?hostsFilter=del HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example request using the `RPP-Authorization` header field for an object that has attached authorisation information:

```
GET /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu
```

Example Info response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 424
Content-Type: application/rpp+json
Content-Language: en
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.6. Create Resource

The client MUST use the HTTP POST method on a resource identifying a collection of object instances (Rule 2, `create` operation).

Example Domain Create request for a new domain name `foo.example`:

```
POST /domainNames HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 220

TODO
```

Example Domain Create response for a new domain name `foo.example`:

```
HTTP/1.1 201 Created
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Language: en
Content-Length: 642
Content-Type: application/rpp+json
Location: https://rpp.example/rpp/domainNames/foo.example
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

Example Domain Create response where a `createProcess` object was implicitly created:

```
HTTP/1.1 201 Created
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Language: en
Content-Type: application/rpp+json
Location: https://rpp.example/rpp/domainNames/foo.example
Link: <https://rpp.example/rpp/domainNames/foo.example/processes/
createProcesses/latest>; rel="rpp-process"; process="createProcess";
processId="XYZ-12345"
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.7. Delete Resource

The client MUST use the HTTP DELETE method on a resource identifying a unique object instance (Rule 2, `delete` operation).

Example Domain Delete request:

```
DELETE /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example Domain Delete response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 80
Content-Type: application/rpp+json
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.8. Update Resource

RPP supports two complementary update operations for modifying an existing object instance, each with its own semantics and use cases:

- **Full update** (HTTP PUT): The client sends a complete replacement representation of the object. The server MUST replace the stored object with the provided representation. Any attributes not present in the request content MUST be treated as absent and cleared or reset to their default values, subject to server policy. The client MUST send all read-write attributes required by the data model, not just the changed ones. The client MUST NOT send any create-only attributes.
- **Partial update** (HTTP PATCH): The client sends only the attributes to be modified. The server MUST apply only the changes indicated in the request content and leave all other attributes unchanged. Data representation of the partial update payload determines how the changes are transmitted between client and server and applied to the data object. The client MAY send changes to any read-write attributes defined in the data model. The client MUST NOT send any create-only attributes.

Both operations MUST be performed on a URL identifying a unique object instance (Rule 2). The request content MUST contain a valid object representation in the negotiated media type.

The server MUST respond with HTTP status code 200 (OK) and include the updated object representation in the response content.

Example full update request (PUT):

```
PUT /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 252
```

Example full update response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 80
Content-Type: application/rpp+json
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

Example partial update request (PATCH):

```
PATCH /domainNames/foo.example HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 252
```

Example partial update response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 80
Content-Type: application/rpp+json
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.9. Processes

Each provisioning object may be related to one or more running processes, such as a transfer or renewal. Each process has its own data, distinct from the data of the provisioning object itself, and may be interacted with using its own set of operations.

All process resources MUST exist under the `/{collection}/{id}/processes/{process-collection}` path, where `{process-collection}` is derived per Rule 3.

16.9.1. Relation to Object Representation

A uniform interface operation MAY require process data in addition to the object representation data. How the process data is embedded in the request content MUST be defined in the corresponding representation specification.

16.9.2. Response with Information about Created Process

When the server creates a process object as a side effect of the operation, it MUST signal this to the client as described in Section 15.

16.9.3. Restore Resource

The Restore operation manages the recovery of an object that has entered the Redemption Grace Period (RGP) [RFC3915]. Restore is modelled as a Process Object with its own lifecycle. The `restoreProcess` object identifier yields the `restoreProcesses` collection segment per Rule 3.

The client MUST use the HTTP POST method to create a new restore process (Rule 4, `create`). The server MUST reject the request with HTTP status code 409 (Conflict) and RPP result code `02304` if the object is not in the `rgpRedemptionPeriod` state. If the server exposes the created restore process resource (see Rule 4), the response MUST include the `Location` header field for that resource.

Example Domain Restore request:

```
POST /domainNames/foo.example/processes/restoreProcesses HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
RPP-Cltrid: ABC-12345
Accept-Language: en
Content-Length: 96
```

Example Restore response:

```
HTTP/1.1 201 Created
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Language: en
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
Content-Length: 85
Location: https://rpp.example/rpp/domainNames/foo.example/processes/
restoreProcesses/XYZ-12345
Content-Type: application/rpp+json
RPP-Code: 01000
```

To submit a restore report for an object in the `rgpPendingRestore` state, the client MUST use the HTTP PUT or PATCH method to update the restore process (Rule 4, `update`). The server MUST reject the request with HTTP status code 409 (Conflict) and RPP result code `02304` if the object is not in the `rgpPendingRestore` state.

Example Domain Restore Report request:

```
PUT /domainNames/foo.example/processes/restoreProcesses/latest HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
RPP-Cltrid: ABC-12346
Accept-Language: en
Content-Length: 180
```

To retrieve the current state of the restore process, the client MUST use the HTTP GET method (Rule 4, `read`) and MUST NOT send content in the request.

16.9.4. Renew Resource

Renew is modelled as a Process Object with its own lifecycle. The `renewProcess` object identifier yields the `renewProcesses` collection segment per Rule 3.

The client MUST use the HTTP POST method to create a new renew process (Rule 4, `create`).

Not every object resource includes support for the renew command. If the server exposes the created renew process resource (see Rule 4), the response MUST include the `Location` header field for that resource.

Example Domain Renew request:

```
POST /domainNames/foo.example/processes/renewProcesses HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
RPP-Cltrid: ABC-12345
Accept-Language: en
Content-Length: 96
```

Example Renew response:

```
HTTP/1.1 201 Created
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Language: en
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
Content-Length: 85
Location: https://rpp.example/rpp/domainNames/foo.example/processes/
renewProcesses/XYZ-12345
Content-Type: application/rpp+json
RPP-Code: 01000
```

> TODO: See https://github.com/ietf-wg-rpp/draft-ietf-rpp-data-objects/issues/195 - read for renew?

16.9.5. Transfer Resource

The Transfer operation manages the change of sponsoring client for a provisioned object. Transfer is modelled as a Process Object with its own lifecycle. The `transferProcess` object identifier yields the `transferProcesses` collection segment per Rule 3.

16.9.5.1. Start

The initiating client MUST use the HTTP POST method to create a new transfer process (Rule 4, `create`).

Example request not using object authorisation:

```
POST /domainNames/foo.example/processes/transferProcesses HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 320
```

Example request using object authorisation:

```
POST /domainNames/foo.example/processes/transferProcesses HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
RPP-Cltrid: ABC-12345
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu
Accept-Language: en
Content-Length: 320
```

Example Transfer response:

```
HTTP/1.1 201 Created
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Language: en
Content-Length: 182
Content-Type: application/rpp+json
Location: https://rpp.example/rpp/domainNames/foo.example/processes/
transferProcesses/latest
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01001
```

16.9.5.2. Status

A transfer process resource may not exist when no transfer has been initiated for the specified object. The client MUST use the HTTP GET method and MUST NOT send content in the request.

Example domain name Transfer Status request:

```
GET /domainNames/foo.example/processes/transferProcesses/latest HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example Transfer Status response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 230
Content-Type: application/rpp+json
Content-Language: en
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.9.5.3. Cancel

The initiating client cancels its pending transfer request using the DELETE method on the process resource (Rule 4, `delete`).

Example request:

```
DELETE /domainNames/foo.example/processes/transferProcesses/latest HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 80
Content-Type: application/rpp+json
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.9.5.4. Reject

The currently sponsoring client rejects a pending transfer. This is an extended process operation; the operation identifier `transferReject` is used unchanged as the final path segment (Rule 5).

Example request:

```
POST /domainNames/foo.example/processes/transferProcesses/latest/
transferReject HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 0
```

Example response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 80
Content-Type: application/rpp+json
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.9.5.5. Approve

The currently sponsoring client approves a pending transfer. The operation identifier `transferApprove` is used unchanged as the final path segment (Rule 5).

Example request:

```
POST /domainNames/foo.example/processes/transferProcesses/latest/
transferApprove HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 0
```

Example response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 80
Content-Type: application/rpp+json
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01000
```

16.10. Messages

The messages endpoint exposes a server-side durable queue of asynchronous notifications generated for a client, for example to report the completion of an operation that was processed out-of-band, such as a Transfer request requiring approval from another party. The server MUST retain a message in the queue until it has been explicitly acknowledged by the client. Retrieval and acknowledgement of a message are deliberately separate operations, using a separate resource and HTTP method for each: the client uses the HTTP GET method to retrieve one or more messages, and the HTTP DELETE method, addressed at a specific message, to acknowledge it. A message that is retrieved but never acknowledged MUST remain in the queue and MUST be made available for redelivery, as described below, so that every message is guaranteed to eventually be delivered to, and can be processed by, the client, subject to the server-initiated deletion described below.

Each message is linked to a single organisation. Only the organisation to which a message is linked is permitted to retrieve or acknowledge that message; the server MUST NOT return a message to, or accept an acknowledgement of a message from, any other organisation.

Notwithstanding the guarantees described in this section, the server MAY delete a message from the queue at any time, including before it has been acknowledged, for example to enforce a retention policy or reclaim storage. Deletion of unacknowledged messages is therefore a best-effort guarantee and clients SHOULD NOT rely on the eventual delivery of every message for correctness.

The server MAY include the `RPP-Queue-Size` header field (see Section 6.2) in any RPP response, not only in responses to Messages requests, to inform the client of the current number of unacknowledged messages in its queue without requiring a dedicated Retrieve request.

Each message follows a lifecycle, as depicted in Figure 1 below, and has a status of either `queued`, `delivered` or `removed`. A message is `queued` from the moment it is created until it is returned to a client in a Retrieve response, at which point its status changes to `delivered`. The server MUST associate an acknowledge timeout (i.e., the maximum time the client has to acknowledge the message) with a message when its status changes to `delivered`. If the client does not acknowledge the message before this timeout elapses, the server MUST revert its status back to `queued`, making it available again for retrieval, including by a different client or reader, as a retry mechanism for a client that crashed or otherwise failed to acknowledge the message. Because of this retry mechanism, a message may be delivered more than once and client message processing MUST be idempotent. Choosing an appropriate delivery timeout duration is an implementation and deployment decision and is out of scope for this document.

The following diagram illustrates the message state transitions described above, including the deletion of a message by the client or server which is permitted while the message is in either the `queued` or the `delivered` state:

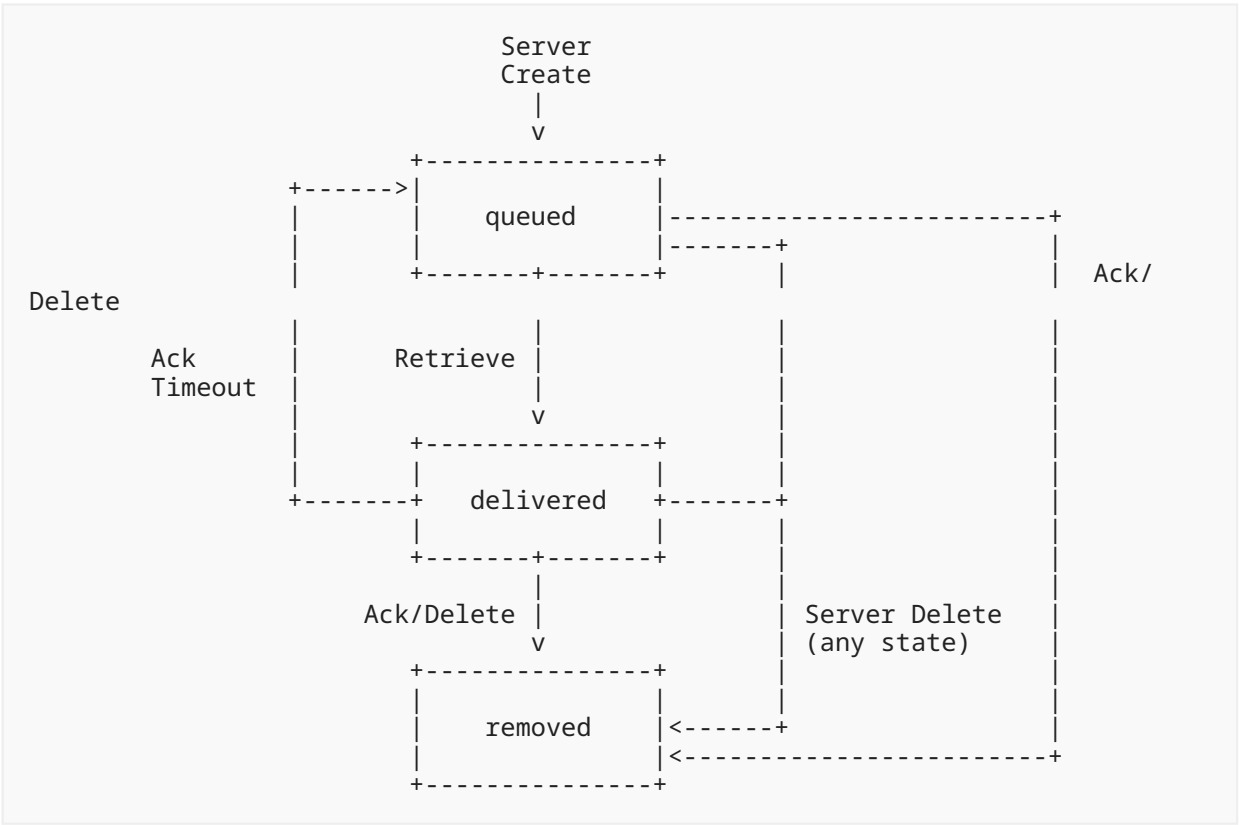


Figure 1: Message Data Object lifecycle

The server MAY support multiple simultaneous readers concurrently retrieving messages from the same queue, for example multiple worker processes operated by the same client. The server MUST NOT return the same `queued` message to more than one Retrieve request at a time; once a message has been returned to a reader and its status changes to `delivered`, it MUST NOT be returned again to any reader until its delivery timeout has elapsed and its status has reverted to `queued`.

The server MUST return `queued` messages in the order in which they were inserted into the queue, so that the oldest `queued` message is always the next one returned to a Retrieve request. A message that reverts from `delivered` back to `queued` after its delivery timeout has elapsed MUST be treated, for ordering purposes, according to its original insertion order rather than being moved to the end of the queue.

16.10.1. Retrieve

The messages endpoint is used for retrieving exactly one message stored on the server for the client to process. The client may use the following query parameter to control which messages are returned:

- `type` (OPTIONAL): Restricts the response to messages of the given message type. This parameter MAY be repeated to request messages of more than one type. If omitted, messages of any type MAY be returned.

Every message has a `type`, identifying the kind of notification it represents. When the client uses the `type` query parameter, only `queued` messages matching one of the requested types transition to `delivered` and are included in the response. A `queued` message that does not match the requested type MUST remain in the queue with a status of `queued`, even if it is ahead, per the ordering rules above, of a matching message that is returned; the `type` filter does not otherwise change the relative position of a message in the queue.

Query parameters are used here for simplicity. A future revision of this document may instead, or additionally, define an equivalent request format using the HTTP QUERY method [RFC10008], which would allow more expressive filtering to be conveyed in the request content rather than the request URL.

The server SHOULD return the human-readable content of a message in the language requested by the client's `Accept-Language` header field. Not every message type may support every language; if the requested language is not available for a given message, the server MUST fall back to returning that message in its own default language.

Every message returned in the response transitions from `queued` to `delivered` as described above.

In the EPP Compatibility Profile, the following limitations apply:

- The RPP server always returns the oldest message in the EPP poll queue.
- Multiple sequential requests from the same client return the same message until the message is acknowledged.
- The `type` query parameter is ignored. The server always returns the oldest message in the EPP poll queue, regardless of its type.

- When multiple readers are used, each reader independently receives the oldest message in the EPP poll queue. Consequently, multiple readers may receive the same message, possibly multiple times, until that message is acknowledged.
- The messages remain in the `queued` state.

Example request for retrieving a new message of the type `transferRequestMessage` only:

```
GET /messages?type=transferRequestMessage HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
Content-Length: 312
Content-Type: application/rpp+json
Content-Language: en
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-Code: 01301
RPP-Queue-Size: 4
```

16.10.2. Acknowledge

The client MUST use the HTTP DELETE method to acknowledge receipt of a single message from the queue. The `id` attribute of a received message MUST be included in the message resource URL, using the `{id}` path element. Once successfully acknowledged, the message MUST be permanently removed from the queue. The server MUST NOT send content in the response to a successful delete request. When the delete request fails for any reason, the server MUST include a Problem Detail document in the content of the error response.

Example request:

```
DELETE /messages/12345 HTTP/1.1
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/1.1 200 OK
Date: Wed, 24 Jan 2024 12:00:00 GMT
Server: Example RPP server v1.0
RPP-Code: 01000
RPP-Queue-Size: 0
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
Content-Length: 0
```

17. Extension Framework

TODO

18. Result Codes

RPP result codes are used to indicate the outcome of an RPP request, they are defined to be backwards compatible with the EPP result codes defined in [RFC5730] and other EPP specifications and extensions.

Two distinct namespaces are defined for RPP result codes:

- Result codes derived from EPP [RFC5730] and other EPP specifications and extensions, which use a leading digit of `0`.
- RPP-specific result codes, which use a leading digit of `1`.

RPP result codes follow the theory of reply codes described in Section 4.2.1. An RPP result code consists of five decimal digits, each digit has a special significance:

- The first digit identifies the namespace, `1` for RPP-specific result codes.
- The second digit denotes the success or failure of the request, see the classes of result codes below.
- The third digit denotes the category of the result, such as syntax or security.
- The fourth and fifth digits provide the explicit result detail within the category.

The second digit has two possible values:

- 1: Positive completion. The request was accepted and processed by the server without error.
- 2: Negative completion. The request was not accepted, and the requested action did not occur.

The third digit groups the results into one of six categories:

- 0: Protocol syntax
- 1: Implementation-specific rules
- 2: Security
- 3: Data management
- 4: Server system

- 5: Connection management

For example, the RPP result code `02303` is an EPP-derived (0) failure (2) in the data management category (3), with the detail "Object does not exist" (03).

Every RPP result code defined in this document and in future standardised RPP extensions MUST be registered with IANA to ensure uniqueness and avoid conflicts. An IANA registry for RPP result codes is defined in the IANA Considerations section.

Newly defined RPP result codes, for example in future extensions, MUST NOT use the namespace reserved for result codes derived from EPP.

18.1. Mapping RPP Result Codes to HTTP Status Codes

RPP result codes are mapped to HTTP status codes, allowing clients to handle responses using common HTTP patterns. The RPP result code, describing the specific outcome of an RPP process, MUST always be returned using the `RPP-Code` header field. This enables the client to take appropriate action based on the detailed result code.

The mapping strategy is to map an RPP result code to the most applicable HTTP status code, when no such mapping can be defined then the RPP result code MUST be mapped to a general HTTP status code (e.g. 200, 400, and 500).

The second and third digits of an RPP result code MUST be used to map the result code to a class of HTTP status codes, as defined in Table 8. The leading namespace digit and the last two detail digits do not influence the class.

RPP Result Code	Meaning	HTTP Status Code Class
x1yzz	Success response	2xx (Successful)
x20zz	Error response, protocol syntax	4xx (Client Error)
x21zz	Error response, implementation-specific rules	4xx (Client Error)
x22zz	Error response, security	4xx (Client Error)
x23zz	Error response, data management	4xx (Client Error)
x24zz	Error response, server system	5xx (Server Error)
x25zz	Error response, connection management	4xx (Client Error) or 5xx (Server Error)

Table 8: Mapping RPP Result Code classes to HTTP Status Code classes

An RPP server MUST NOT use a 1xx or 3xx HTTP status code to signal an RPP result code. Redirection is handled as described in Section 8.

Table 9 lists the RPP result codes and their mapping to specific HTTP status codes, any RPP result code not listed in the table MUST be mapped to a generic HTTP status code as defined in Table 10.

RPP Result Code	HTTP Status Code	Description
x1000	200 (OK)	Request completed successfully
02002	409 (Conflict)	Command use error
02101	501 (Not Implemented)	Unimplemented command
02102	501 (Not Implemented)	Unimplemented option
02103	501 (Not Implemented)	Unimplemented extension
02104	402 (Payment Required)	Billing failure
02105	409 (Conflict)	Object is not eligible for renewal
02106	409 (Conflict)	Object is not eligible for transfer
02200	401 (Unauthorized)	Authentication error
02201	403 (Forbidden)	Authorization error
02202	401 (Unauthorized)	Invalid authorization information
02300	409 (Conflict)	Object pending transfer
02301	409 (Conflict)	Object not pending transfer
02302	409 (Conflict)	Object exists
02303	404 (Not Found)	Object does not exist
02304	409 (Conflict)	Object status prohibits operation
02305	409 (Conflict)	Object association prohibits operation
02306	422 (Unprocessable Content)	Parameter value policy error
02307	501 (Not Implemented)	Unimplemented object service
02308	422 (Unprocessable Content)	Data management policy violation
02501	401 (Unauthorized)	Authentication error; server closing connection
02502	429 (Too Many Requests)	Session limit exceeded; server closing connection

Table 9: Mapping RPP Result Codes to specific HTTP Status Codes

The generic class-based mapping in Table 10 is used for all other RPP result codes.

RPP Result Code	HTTP Status Code	Description
x1yzz	200 (OK)	Command completed successfully
x20zz	400 (Bad Request)	Protocol syntax error
x21zz	400 (Bad Request)	Implementation-specific rule error
x22zz	400 (Bad Request)	Security error
x23zz	400 (Bad Request)	Data management error
x24zz	500 (Internal Server Error)	Server system error
x25zz	500 (Internal Server Error)	Connection management error, used when no specific mapping is listed

Table 10: Mapping RPP Result Codes to generic HTTP Status Codes

19. Authentication and Authorisation

Due to the stateless nature of RPP, the client MUST include the authentication credentials in each HTTP request. This MAY be done by using JSON Web Tokens (JWT) [RFC7519] or Basic authentication [RFC7617]. The server MUST validate the authentication credentials on each request and reject any request with invalid credentials with an appropriate HTTP status code.

When using JWTs for OAuth 2.0 [RFC6749] Access Tokens, the JWT profile described in [RFC9068] MUST be used. It is RECOMMENDED to use short-lived tokens and to implement token revocation mechanisms to mitigate the risk of token compromise. If sensitive information is included in the JWT payload, it MUST be encrypted to prevent unauthorised access when the token is persisted to a storage device. Furthermore, the best practices for JWT usage as outlined in [RFC8725] MUST be followed.

20. IANA Considerations

20.1. RPP Result Codes Registry

The IANA is requested to create a new registry "RPP Result codes", this registry will be used to register RPP result codes defined in this document and in future RPP specifications and extensions.

```
Name of the registry: RPP Result codes
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `code`: The RPP result code, for example `01000`.

- `origin`: The origin of the result code, indicating whether it is derived from EPP or defined specifically for RPP.
- `description`: A human-readable description of the result code and its intended use.

The initial set of codes is listed in the table below.

RPP Result Code	Origin	Description
01000	EPP	Command completed successfully
01001	EPP	Command completed successfully; action pending
01300	EPP	Command completed successfully; no messages
01301	EPP	Command completed successfully; ack to dequeue
01500	EPP	Command completed successfully; ending session
02000	EPP	Unknown command
02001	EPP	Command syntax error
02002	EPP	Command use error
02003	EPP	Required parameter missing
02004	EPP	Parameter value range error
02005	EPP	Parameter value syntax error
02100	EPP	Unimplemented protocol version
02101	EPP	Unimplemented command
02102	EPP	Unimplemented option
02103	EPP	Unimplemented extension
02104	EPP	Billing failure
02105	EPP	Object is not eligible for renewal
02106	EPP	Object is not eligible for transfer
02200	EPP	Authentication error
02201	EPP	Authorization error
02202	EPP	Invalid authorization information
02300	EPP	Object pending transfer
02301	EPP	Object not pending transfer
02302	EPP	Object exists
02303	EPP	Object does not exist
02304	EPP	Object status prohibits operation

RPP Result Code	Origin	Description
02305	EPP	Object association prohibits operation
02306	EPP	Parameter value policy error
02307	EPP	Unimplemented object service
02308	EPP	Data management policy violation
02400	EPP	Command failed
02500	EPP	Command failed; server closing connection
02501	EPP	Authentication error; server closing connection
02502	EPP	Session limit exceeded; server closing connection

Table 11: Initial RPP Result Codes registrations

20.2. Link Relation Type

The IANA is requested to register the "rpp-process" link relation type in the "Link Relation Types" registry, following the template in [RFC8288]:

```
Relation Name: rpp-process
Description:   Identifies a process resource that was implicitly created as a
               side effect of a uniform interface operation on the target
               resource.  The context resource is the provisioning object on
               which the operation was performed; the target resource is the
               created process instance.  The following target attributes are
               defined for this relation type: "process" (the Process Object
               Identifier, required), "processId" (the server-assigned
process
               instance identifier, optional), and "latest" (the boolean
value
               "true", optional, indicating the target URL uses the "latest"
               mnemonic).
Reference:     This document
```

21. Internationalisation Considerations

TODO

22. Security Considerations

RPP relies on the security of the underlying HTTP transport, hence the best common practices for securing HTTP described in [RFC9325] also apply to RPP and MUST be followed by RPP implementations.

Data confidentiality and integrity MUST be enforced. Every client and server interaction MUST be encrypted using TLS version 1.3 [RFC8446]. Future versions of TLS MAY be used as they become available and are deemed secure.

RPP does not mandate a single data format; media types for RPP messages can use JSON, YAML, or any other data format. Each RPP media type specification MUST document the security considerations applicable to its underlying format (e.g. [RFC8259] for JSON), in addition to the considerations described in this section. An implementation that infers the data format or profile of a message from its content, rather than from the `Content-Type` header field, risks processing the message with the wrong parser or feature set; Section 13 therefore requires both RPP clients and RPP servers to rely on the header field only.

An RPP client MUST verify the identity of the RPP server, as described in Section 4.3.4, before sending any request. The client MUST validate the TLS server certificate, including that the certificate chain is trusted and that the certificate matches the host name of the URL used to access the RPP server. If the validation fails, the client MUST NOT continue and MUST NOT send the request.

23. Change History

23.1. Version ietf-rpp-core-01 to ietf-rpp-core-02

23.2. Version ietf-rpp-core-00 to ietf-rpp-core-01

- Added generic mapping of operation parameters to HTTP query parameters in Section 16.1.7 (Issue #135)
- Added Section 8 (Issue #133)
- Added TLS server certificate validation requirements to Section 22 (Issue #133)
- Updated the draft to comply with the HTTP Editorial Style Guide (Issue #103)
- Removed all JSON examples, these are now found in the rpp-json draft (Issue #74)
- Moved specification of discovery and profile data objects and the related IANA registration requests to rpp-data-objects (Issue #125)
- Updated the reference for the HTTP specification (Issue #90)
- Removed hardcoded version segment from paths and added media type and header field profile signalling in Section 14 (Issues #97, #98, #101)
- Updated Section 6.1 and Section 6.2 to use Structured Fields [RFC9651] (Issue #95)
- Added support for Internationalized Domain Names (IDN) in Section 16.3 (Issue #126)
- Added description of service messages to Section 12.2 (Issue #116)
- Consolidated multiple paragraphs into a single Section 18 section (Issue #92)
- Added Section 7 for browser-based clients (Issue #20)
- Removed text suggesting HTTP/2 is minimum version required for RPP (Issue #91)
- Added Section 16.9.3 based on the Restore Process Object (Issue #117)
- Added paths for availability operation and aligned Section 16.4 with the Availability operation defined in rpp-data-objects (Issue #149)
- Fixed spelling errors and harmonised spelling to British English
- Fixed inconsistencies in the HTTP examples
- Updated the requirements language boilerplate to BCP 14, citing RFC 8174, and corrected key word usage

- Made the `Location` header field in Restore and Renew responses conditional on the process resource being exposed
- Corrected the DNS HTTPS resource record examples in Section 10
- Corrected the profile signalling ABNF and the `RPP-Profile` Structured Field type in Section 14
- Reframed the Abstract and Introduction to describe this document as the HTTP mapping of the RPP data model
- Added Section 4
- Added media type negotiation requirements to Section 13
- Added Section 14.1
- Updated the bootstrap step of the Discoverability workflow to use DNS HTTPS resource records (Issue #100)
- Updated the mapping of RPP result codes to HTTP status codes in Section 18
- Removed the URN sub-namespace and `application/rpp+json` media type registrations from IANA Considerations
- Reclassified RFC 5321, RFC 8259, RFC 10008, and REST as informative references

23.3. Version draft-wullink-rpp-core-05 to draft-ietf-rpp-core-00

- Renamed the document name to "draft-ietf-rpp-core" and reset version number to 00 (Issue #80)
- Added Organisation and User resource type, with create, read, update and delete operations. (Issue #67)
- Added section about Mapping to EPP. (Issue #55)
- Described full and partial update operations, using HTTP PUT and PATCH methods respectively. (Issue #21)
- Added support for embedding process data in uniform interface operations. Registered the `rpp-process` link relation type with IANA.
- Generalised process sub-resource URL derivation into Rule 3 (Direct Access Sub-Resource Path Segment); Rules 4 through 6 now apply this general rule to Process Objects instead of deriving process paths independently.
- Corrected the Renew process collection segment from `renewalProcesses` to `renewProcesses` in the Renew Resource section and the Derived Endpoint Reference table, matching the `renewProcess` identifier in [I-D.ietf-rpp-data-objects].
- Aligned the URL template variables in Discoverability with Unique Identifier terminology from [I-D.ietf-rpp-data-objects] and with the `{process-collection}/{process-id}` naming used in the Endpoints section; removed the unused `process_name` variable.

23.4. Version 04 to 05

- Added Bootstrap and Discovery sections to the document, describing how a client can discover the location and capabilities of an RPP server
- Added IANA Considerations section with a request for new RPP discovery URLs, extensions and profile URLs.

23.5. Version 03 to 04

- Added a new section on versioning, describing how versioning is applied to different RPP elements. (Issue #39)
- Added IANA request for RPP Result codes registry, and added a table with example RPP result codes. (Issue #39)
- Added IANA request for URN sub-namespace for RPP. (Issue #39)
- Added a new "Profiles" section to describe how to define and use profiles. (Issue #43)
- Added a new section "Authentication and Authorisation". (Issue #37)
- Updated
the "Security Considerations" section to include transport security. (Issue #37)
- Added IANA registration request for the new RPP media type. (Issue #40)

23.6. Version 02 to 03

- Added use of Problem Detail [RFC9457] for error responses

23.7. Version 01 to 02

- Updated the examples, changed from ".example.org" to ".example"
- Merged the RPP-EPP-Code and RPP-Code headers into a single RPP-Code header
- Update the RPP-Authorization header to match the HTTP Authorization header format
- Added new process path segment and process representations
- Updated the Check request to now use an "availability" path segment and support both GET and HEAD methods

23.8. Version 00 to 01

- Updated "Request Headers" and "Response Headers" section
- Changed transfer resource URL and HTTP method for reject, approve and cancel, in order to make the API easier to use

23.9. Version 00 (draft-rpp-core) to 00 (draft-wullink-rpp-core)

- Renamed the document name to "draft-wullink-rpp-core"
- Removed sections: Design Considerations, Resource Naming Convention, Session Management, HTTP Layer, Content Negotiation, Object Filtering, Error Handling
- Renamed Commands section to Endpoints
- Removed text about extensions
- Changed naming to be less EPP like and more RDAP like

24. Acknowledgements

The authors would like to thank the following people for their helpful text contributions, comments and suggestions.

- Q Misell, AS207960 Cyfyngedig

25. References

25.1. Normative References

[FETCH] WHATWG, "Fetch - Living Standard", <https://fetch.spec.whatwg.org/>.

[HTTP] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <https://www.rfc-editor.org/info/rfc9110>.

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC5730] Hollenbeck, S., "Extensible Provisioning Protocol (EPP)", STD 69, RFC 5730, DOI 10.17487/RFC5730, August 2009, <https://www.rfc-editor.org/info/rfc5730>.

[RFC5731] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Domain Name Mapping", STD 69, RFC 5731, DOI 10.17487/RFC5731, August 2009, <https://www.rfc-editor.org/info/rfc5731>.

[RFC5733] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Contact Mapping", STD 69, RFC 5733, DOI 10.17487/RFC5733, August 2009, <https://www.rfc-editor.org/info/rfc5733>.

[RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", RFC 5890, DOI 10.17487/RFC5890, August 2010, <https://www.rfc-editor.org/info/rfc5890>.

[RFC6570] Gregorio, J., Fielding, R., Hadley, M., Nottingham, M., and D. Orchard, "URI Template", RFC 6570, DOI 10.17487/RFC6570, March 2012, <https://www.rfc-editor.org/info/rfc6570>.

[RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <https://www.rfc-editor.org/info/rfc6749>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/info/rfc7519>.

[RFC7617] Reschke, J., "The 'Basic' HTTP Authentication Scheme", RFC 7617, DOI 10.17487/RFC7617, September 2015, <https://www.rfc-editor.org/info/rfc7617>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/info/rfc8174>.

[RFC8288] Nottingham, M., "Web Linking", RFC 8288, DOI 10.17487/RFC8288, October 2017, <https://www.rfc-editor.org/info/rfc8288>.

[RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, DOI 10.17487/RFC8446, August 2018, <https://www.rfc-editor.org/info/rfc8446>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/info/rfc8725>.

[RFC9068] Bertocci, V., "JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens", RFC 9068, DOI 10.17487/RFC9068, October 2021, <https://www.rfc-editor.org/info/rfc9068>.

[RFC9325] Sheffer, Y., Saint-Andre, P., and T. Fossati, "Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)", BCP 195, RFC 9325, DOI 10.17487/RFC9325, November 2022, <https://www.rfc-editor.org/info/rfc9325>.

[RFC9457] Nottingham, M., Wilde, E., and S. Dalal, "Problem Details for HTTP APIs", RFC 9457, DOI 10.17487/RFC9457, July 2023, <https://www.rfc-editor.org/info/rfc9457>.

[RFC9460] Schwartz, B., Bishop, M., and E. Nygren, "Service Binding and Parameter Specification via the DNS (SVCB and HTTPS Resource Records)", RFC 9460, DOI 10.17487/RFC9460, November 2023, <https://www.rfc-editor.org/info/rfc9460>.

[RFC9535] Gössner, S., Ed., Normington, G., Ed., and C. Bormann, Ed., "JSONPath: Query Expressions for JSON", RFC 9535, DOI 10.17487/RFC9535, February 2024, <https://www.rfc-editor.org/info/rfc9535>.

[RFC9651] Nottingham, M. and P. Kamp, "Structured Field Values for HTTP", RFC 9651, DOI 10.17487/RFC9651, September 2024, <https://www.rfc-editor.org/info/rfc9651>.

[SemVer] Semantic Versioning, "Semantic Versioning 2.0.0", <https://semver.org/>.

25.2. Informative References

[IDN-Tables] Internet Assigned Numbers Authority (IANA), "Repository of IDN Practices", <https://www.iana.org/assignments/idn-tables>.

[REST] Fielding, R., "Architectural Styles and the Design of Network-based Software Architectures", 2000, <https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm>.

[RFC10008] Reschke, J., Snell, J.M., and M. Bishop, "The HTTP QUERY Method", RFC 10008, DOI 10.17487/RFC10008, June 2026, <https://www.rfc-editor.org/info/rfc10008>.

[RFC3915] Hollenbeck, S., "Domain Registry Grace Period Mapping for the Extensible Provisioning Protocol (EPP)", RFC 3915, DOI 10.17487/RFC3915, October 2004, <https://www.rfc-editor.org/info/rfc3915>.

[RFC5321] Klensin, J., "Simple Mail Transfer Protocol", RFC 5321, DOI 10.17487/RFC5321, October 2008, <https://www.rfc-editor.org/info/rfc5321>.

[RFC8259] Bray, T., Ed., "The JavaScript Object Notation (JSON) Data Interchange Format", STD 90, RFC 8259, DOI 10.17487/RFC8259, December 2017, <https://www.rfc-editor.org/info/rfc8259>.

[RFC8748] Carney, R., Brown, G., and J. Frakes, "Registry Fee Extension for the Extensible Provisioning Protocol (EPP)", RFC 8748, DOI 10.17487/RFC8748, March 2020, <https://www.rfc-editor.org/info/rfc8748>.

[RFC9205] Nottingham, M., "Building Protocols with HTTP", BCP 56, RFC 9205, DOI 10.17487/RFC9205, June 2022, <https://www.rfc-editor.org/info/rfc9205>.

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/